

## FastMem Lab — Concept Summary

Claim under test: A fast-weight matrix can store and retrieve associations without gradient descent, but capacity is bounded and new writes interfere with old ones.

Mechanism: A key  $\mathbf{k}$  and value  $\mathbf{v}$  (vectors in  $\mathbb{R}^d$ ) are bound into a  $d \times d$  memory matrix  $\mathbf{W}$  by a Hebbian outer-product write,  $\mathbf{W} \leftarrow \mathbf{W} + \eta(\mathbf{k}\mathbf{v}^T)$ , with no loss function and no backpropagation anywhere in the update. Retrieval is a single matrix-vector product,  $\hat{\mathbf{v}} = \mathbf{W}^T \mathbf{q}$ . This is the same mechanism the fast-weight-programmer literature calls a "correlation matrix memory" (Irie & Gershman, 2025), and it is the mechanism BDH's published architecture (Kosowski et al., 2025) describes as implementing working memory through Hebbian synaptic plasticity.

What the lab lets you do: write associations and watch the actual matrix change; query them back and compare retrieved values against ground truth side by side; run a controlled interference experiment that fixes several early associations as probes and re-queries them as more associations are written into the same fixed-size matrix; sweep learning rate, memory dimension, and query noise; and compare fast-weight retrieval against an unbounded nearest-neighbor baseline.

What the experiments show:

- A single association is retrieved almost perfectly (similarity  $> 0.99$ ).
- As more associations are superimposed into a fixed-size matrix, retrieval quality for the earliest associations measurably degrades — interference, verified by automated tests, not just described.
- A larger memory dimension sustains more associations before that degradation sets in — a direct, testable capacity/dimension relationship.
- Query noise degrades retrieval \*magnitude\* (normalized L2 error) for a single association; cosine similarity alone is not a reliable signal in that specific regime, a real mathematical property this project surfaces rather than glosses over.
- Nearest-neighbor memory does not show the same interference (it stores everything

explicitly), at the cost of unbounded storage — the two mechanisms are presented as a trade-off, not a "winner."

Relation to BDH: Both this project's `W` and BDH's synaptic memory are directly, Hebbian-updated matrices of pairwise associations, distinct from slower gradient-trained parameters. This project is explicitly an independent, simplified educational implementation — not the official BDH codebase, and it makes no claim to reproduce BDH's benchmark results. See `research/bdh-notes.md` for the full comparison.

Honesty: every number in this lab comes from live NumPy computation on the backend, triggered by an actual HTTP request from the frontend. Nothing is scripted, precomputed, or hidden when retrieval fails — see `docs/provenance` classification in the app and `docs/limitations.md`.